THE UNIVERSITY OF MELBOURNE

COMP30023 – Computer Systems

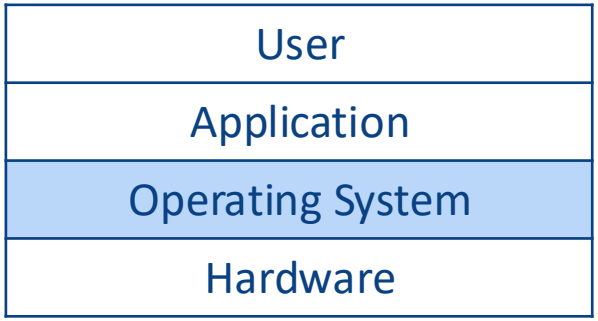
Introduction to Operating Systems

Maria Rodriguez Read

Semester 1, 2025

What is an Operating System?

- An OS is a program that interfaces the machine with the application programs

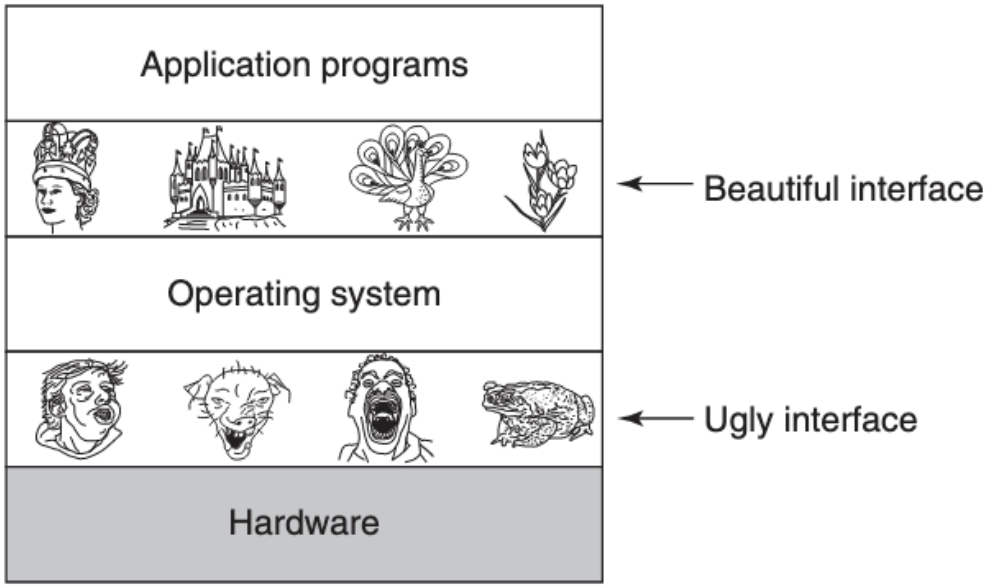


- Its job is to make it so other programs run efficiently
- Key functionality:
  - Provide hardware abstraction
  - Manage hardware resources

© University of Melbourne 2025

Hardware Abstraction

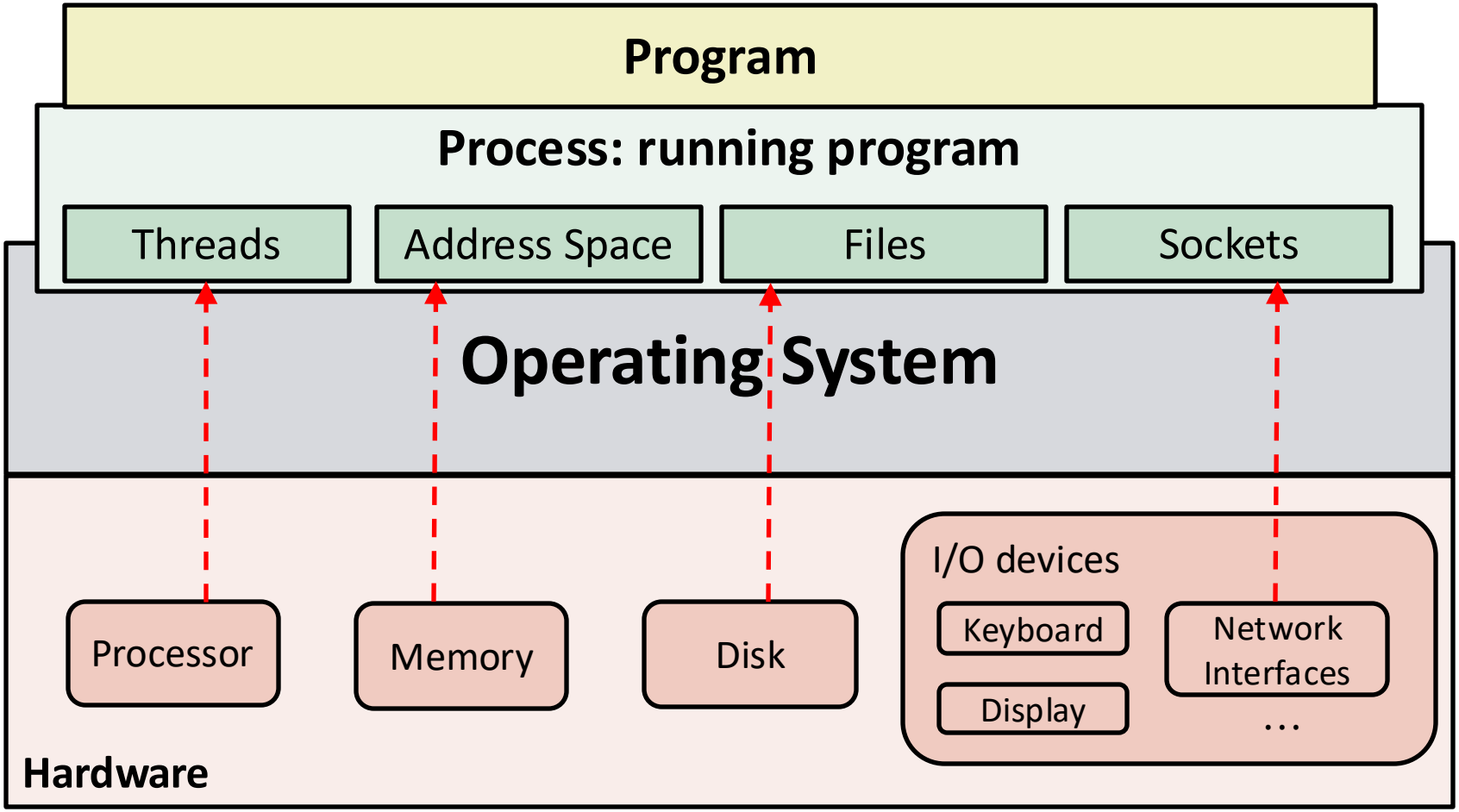
- Two goals:
  - Present a simpler, cleaner, easy to use and understand model of the computer
  - Present an application with a simpler abstract machine that appears to be dedicated to itself



TB: Figure 1-2

© University of Melbourne 2025

Hardware Abstraction



© University of Melbourne 2025

Resource Management

- Provide for an orderly and controlled allocation of the processors, memories, and I/O devices
- Why is this important?
  - Hint:

  % `ps aux` (Mac, Linux)

  % `tasklist` (Windows)
- Resources need to be shared between competing processes!

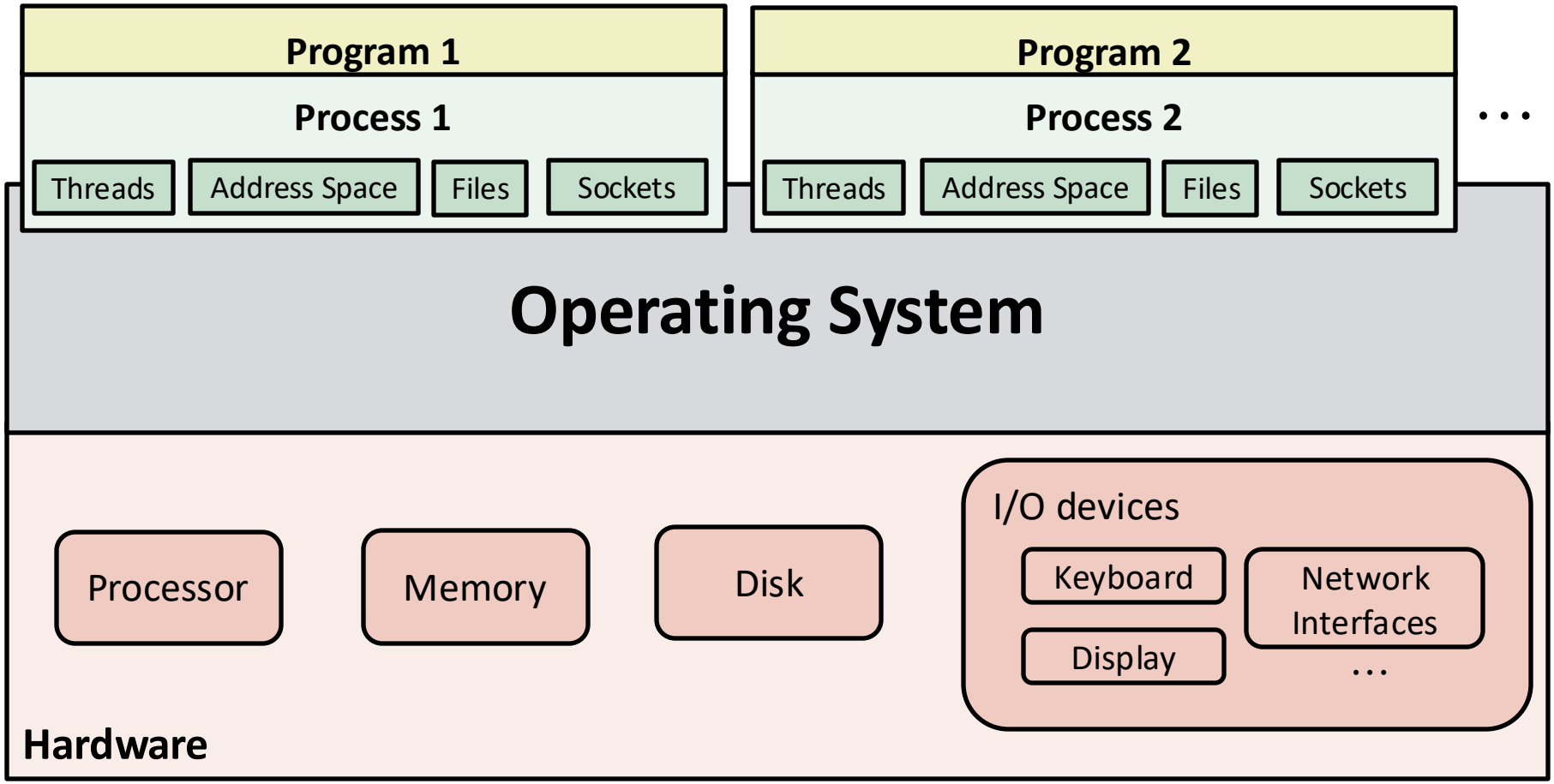
© University of Melbourne 2025

Resource Management

- Provide for an orderly and controlled allocation of the processors, memories, and I/O devices **among all the programs wanting them.**

© University of Melbourne 2025

Resource Management

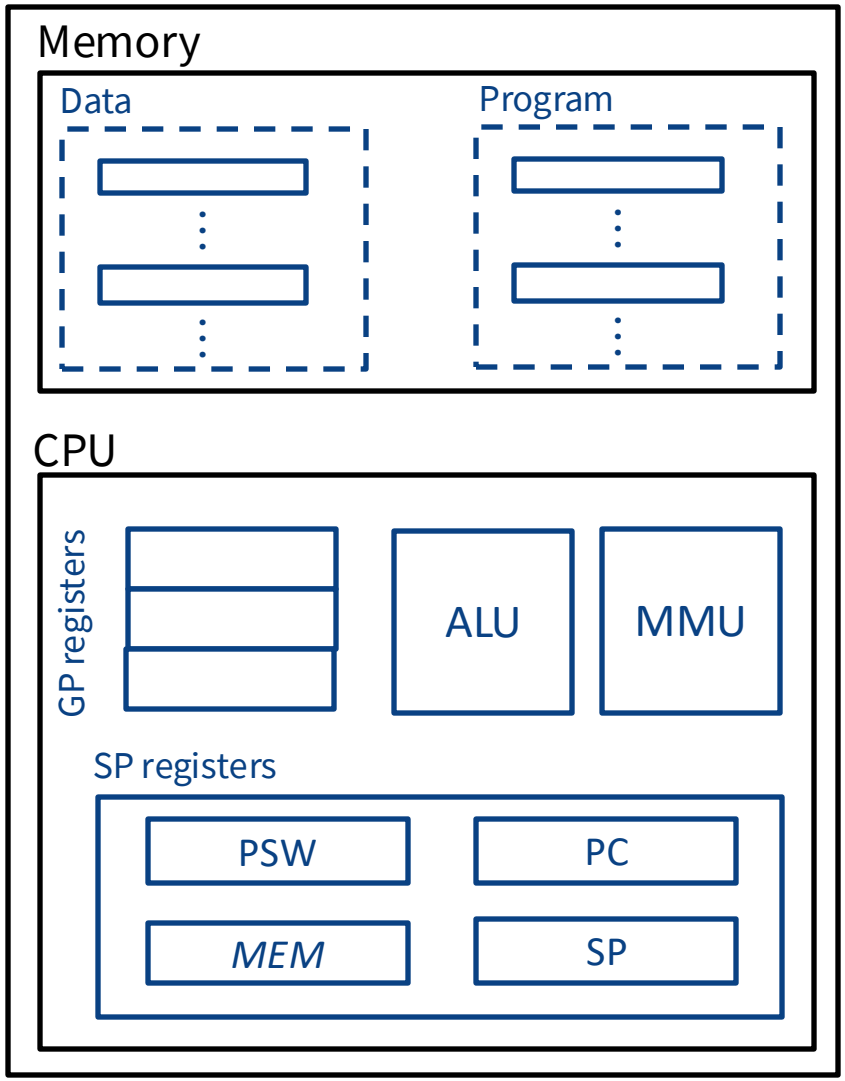


© University of Melbourne 2025

Computer Architecture Background

- Consider the computer hardware as containing 2 main components: memory and CPU
- Memory: where programs and data are stored
- CPU: does the actual computation
  - Registers
    - Fastest tier of memory (Register -> Cache -> Main Memory -> Disk)
    - General-purpose: store data (e.g. int, float) and memory addresses
    - Special-purpose: specific control functions
    - Combination of GP and SP registers is used to support program execution.
  - ALU: arithmetic logic unit
  - MMU: memory management unit
  - Our (simplistic) view: 1 CPU, 1 core, executes one instruction per cycle

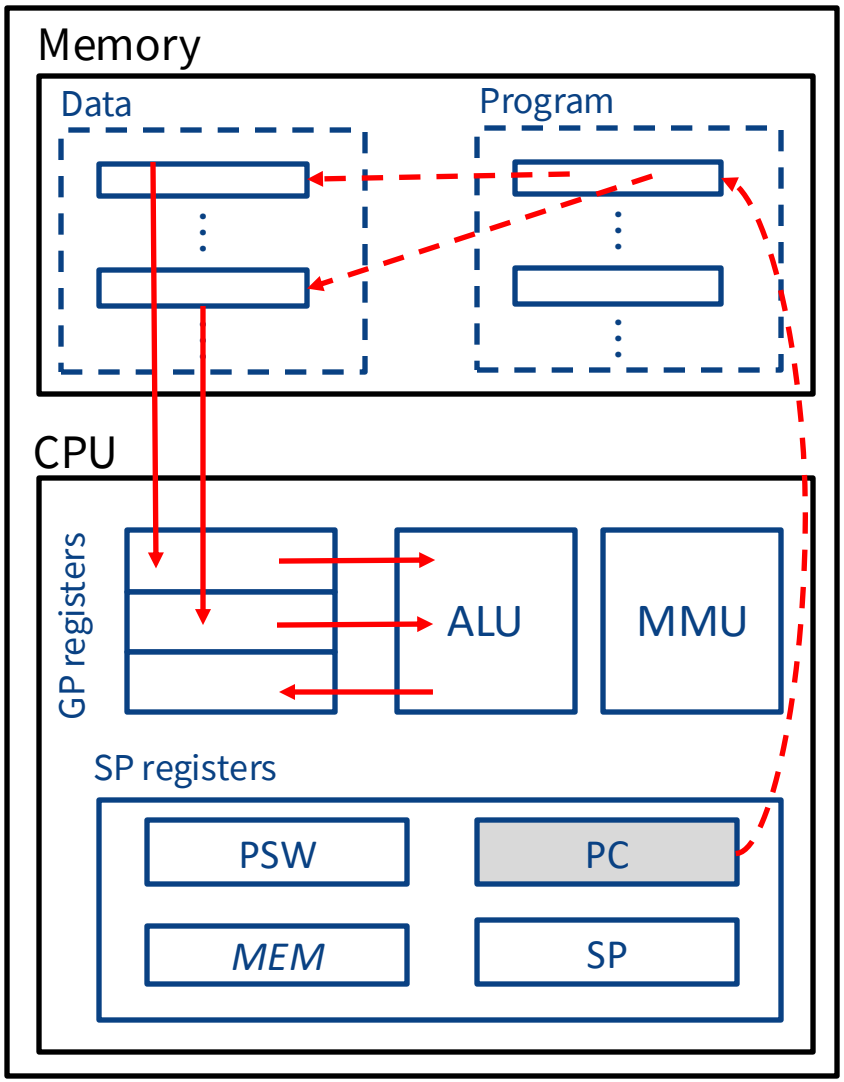
© University of Melbourne 2025



© University of Melbourne 2025

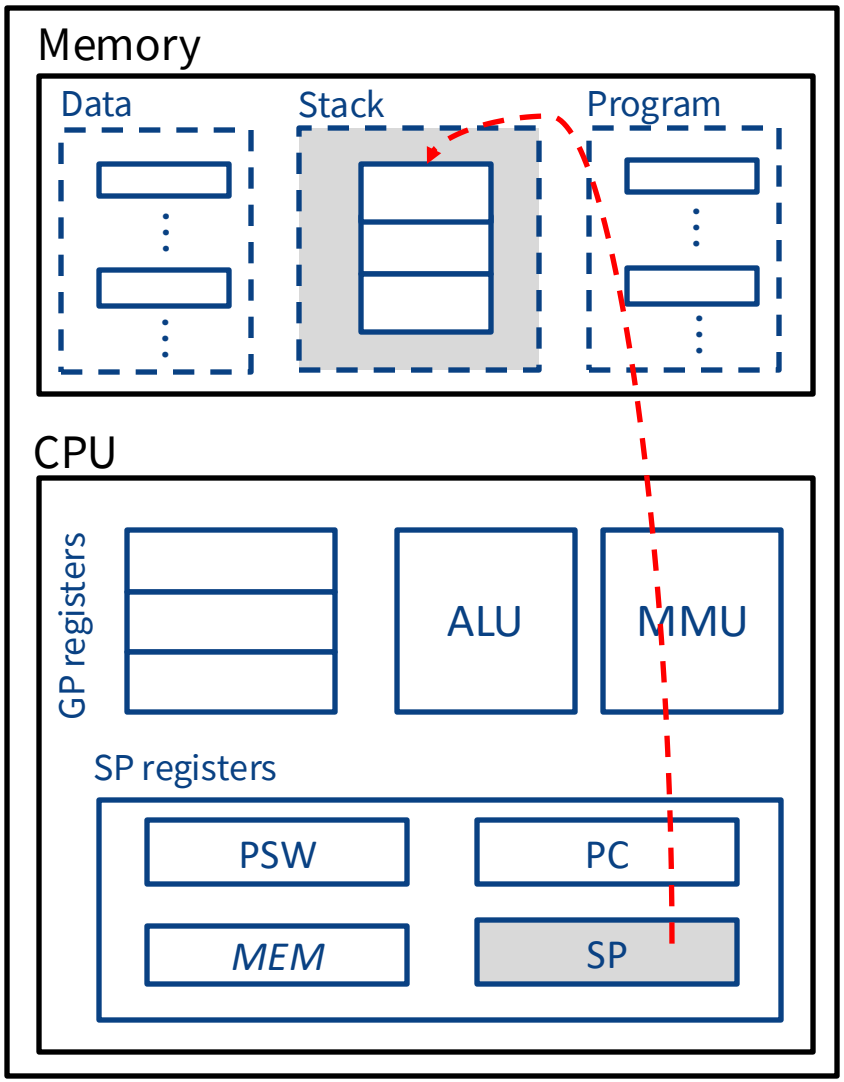
Running an Instruction

- Each cycle, the CPU:
  - Gets an instruction from memory (fetch)
    - Program Counter (PC): keeps track of the memory address of the next instruction to be executed in a program
  - Figures out what the instruction is (decode)
  - Executes the instruction (execute)
    - May include copying operands from memory to registers, using the ALU to perform some operation on these values, and storing the result in another register



Calling Subroutines

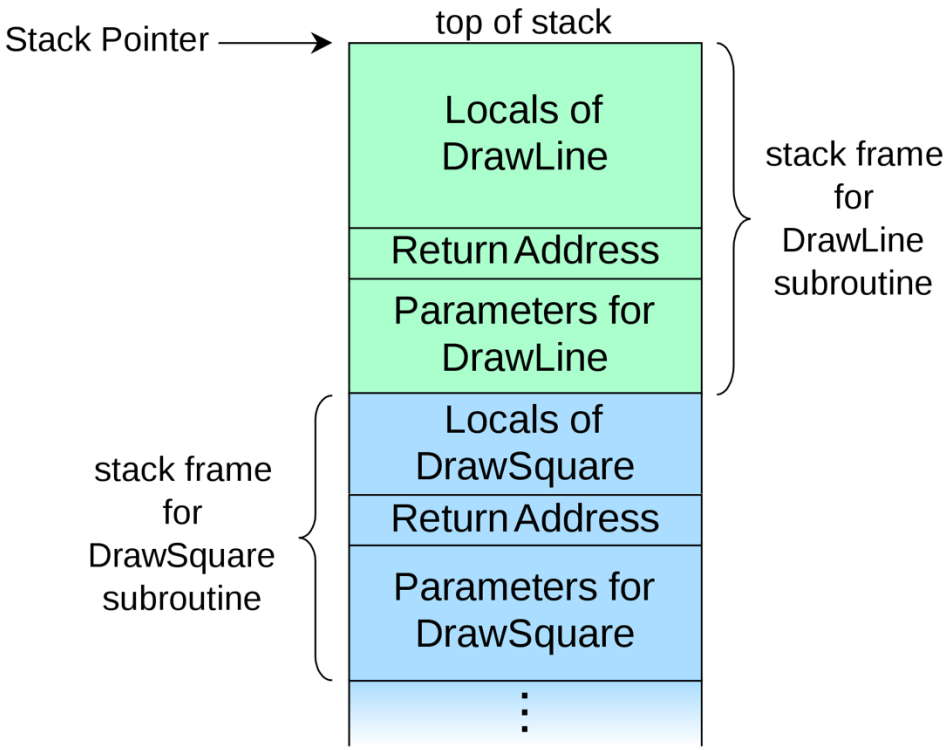
- Subroutine: function called from within another function
- State of a function: local variables + registers
- When a function calls a subroutine, the state of the calling function must be maintained/saved
  - Stack is used for this purpose
- Examples of HW support:
  - `call, ret, push, pop` instructions
  - Stack Pointer register



© University of Melbourne 2025

Stack

- Special region of memory that stores information on active subroutines
- Composed of stack frames
- Stack frame corresponds to a call to a subroutine that has not yet terminated
- Contains (at least) address to return to when subroutine call is complete, input args, local vars
- Stack pointer (SP)
  - Points to the top of the stack



By R. S. Shaw - Own work, Public Domain
https://commons.wikimedia.org/w/index.php?curid=1956587

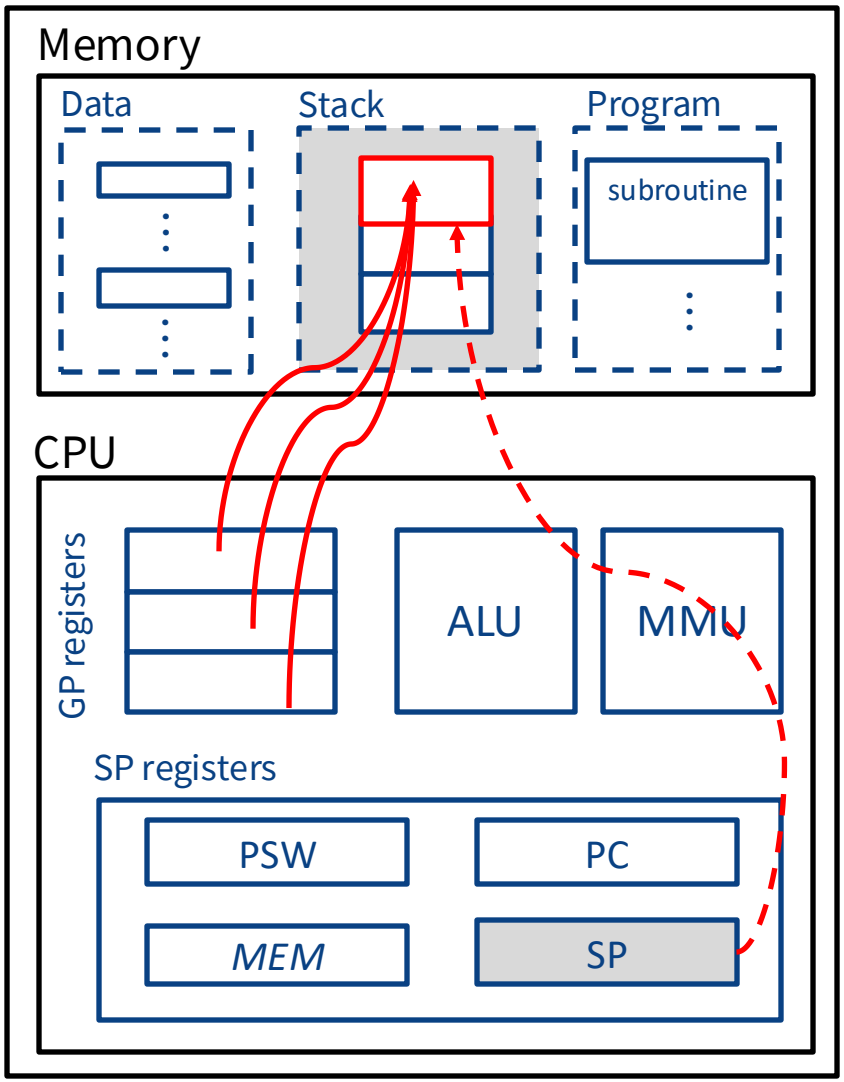
Registers + Stack

- Sample assembly program illustrating the use of registers and stack

© University of Melbourne 2025

Calling a subroutine

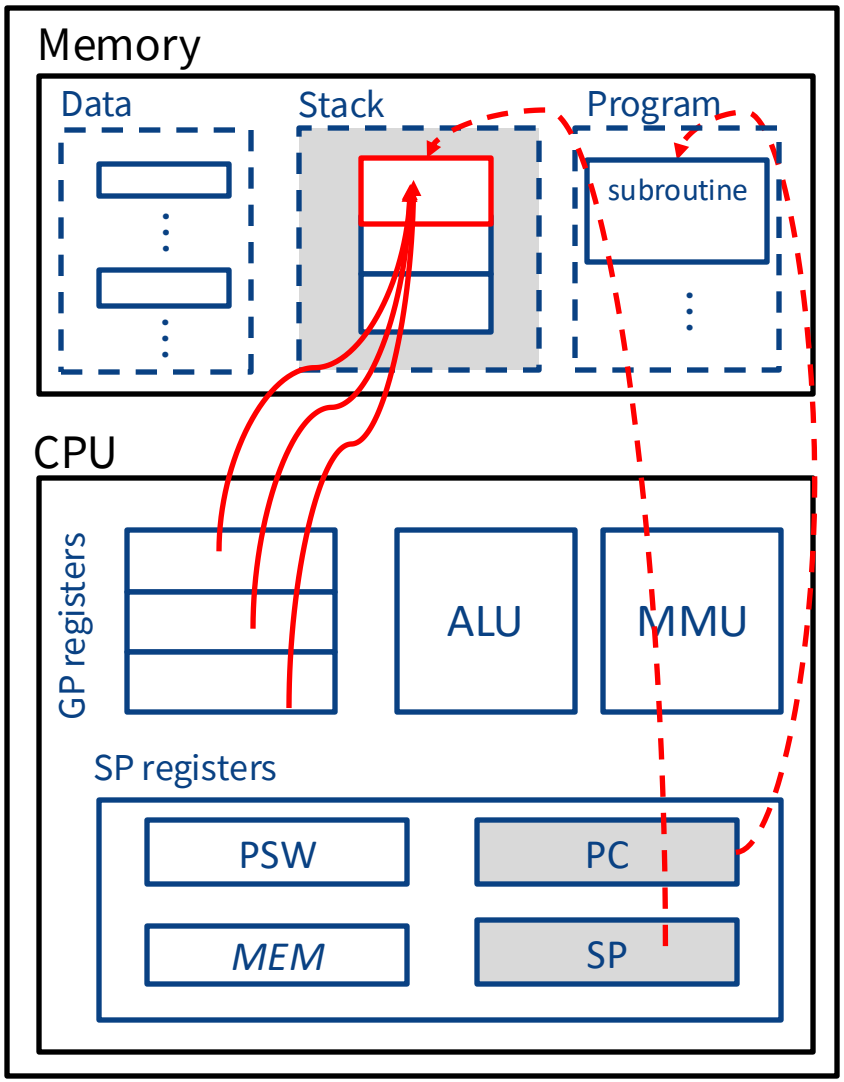
- Sample set of tasks carried out when calling a subroutine:
  - Register values are stored in the stack at the location pointed to by the SP
  - The return address (the address after the call instruction) is stored in the stack



© University of Melbourne 2025

Calling a subroutine (contd.)

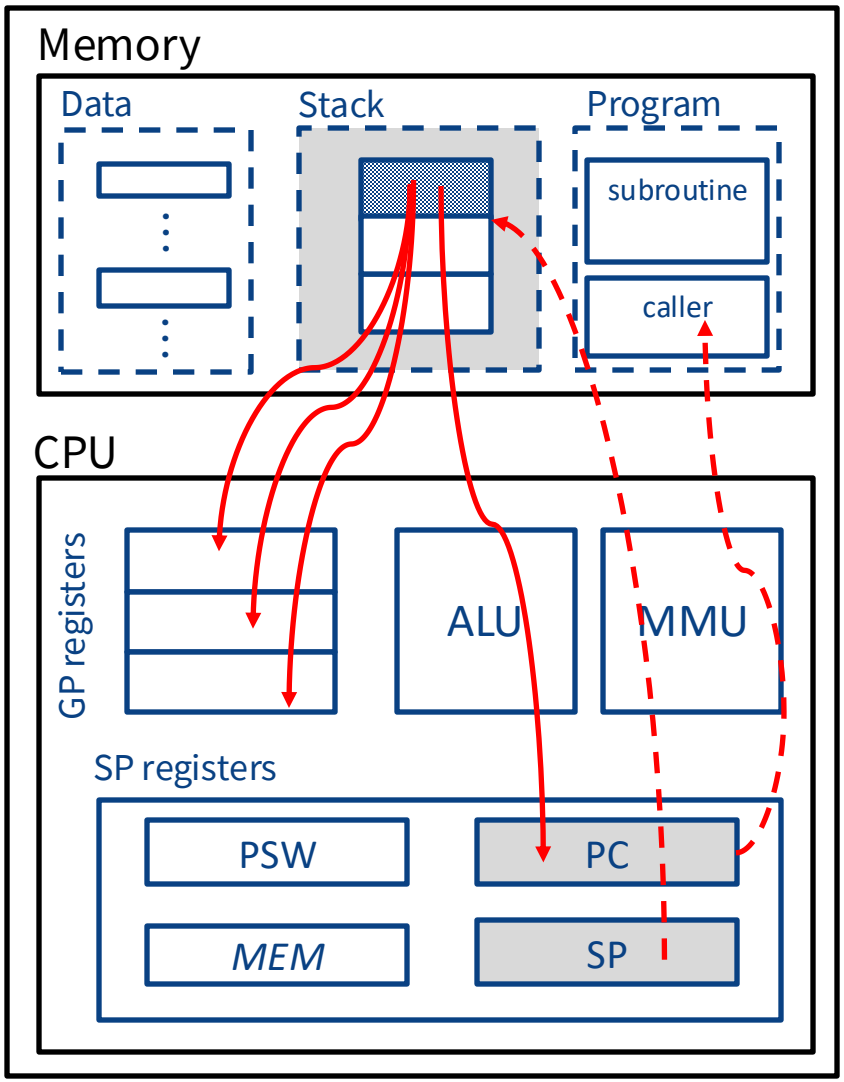
- The PC is loaded with the address of the entry point of the subroutine that has been invoked
- The stack pointer is updated to point to the new top of the stack



© University of Melbourne 2025

Returning from a Subroutine

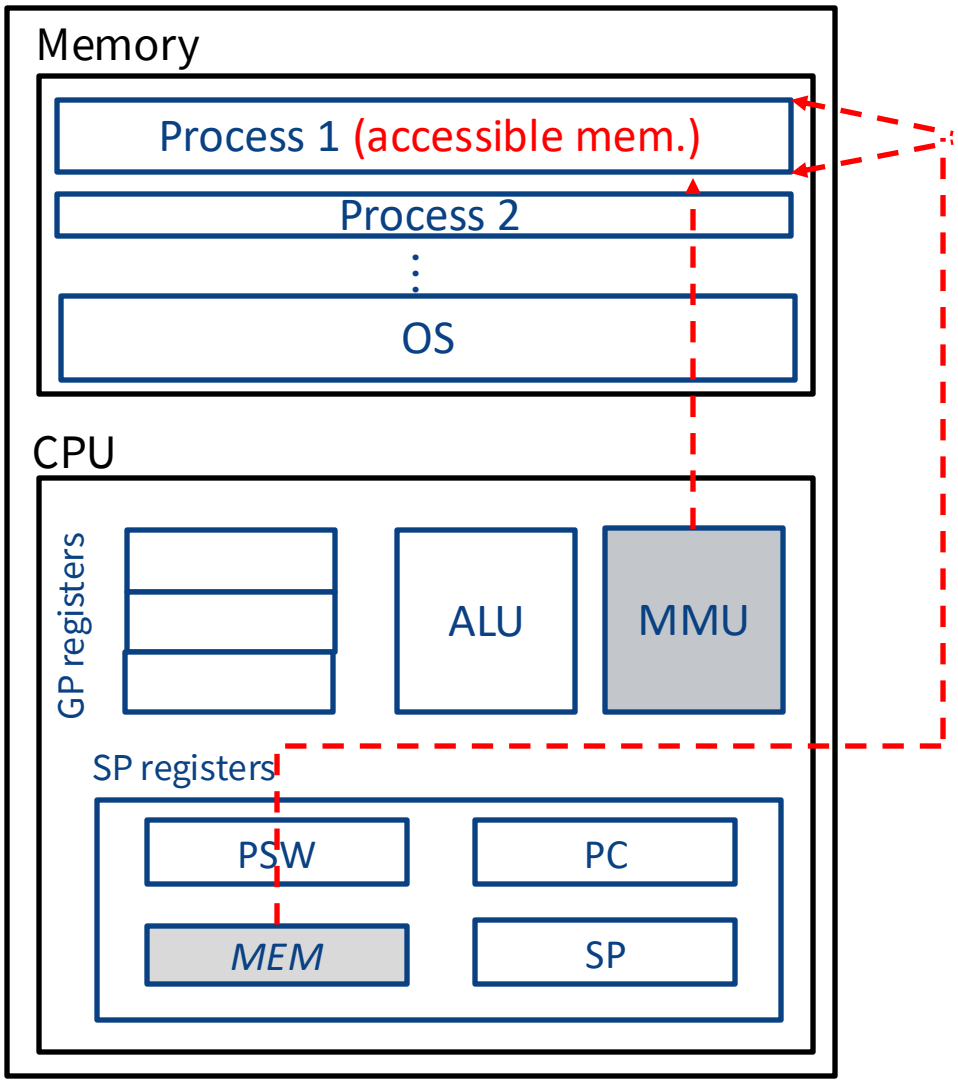
- Sample set of tasks carried out when returning from a subroutine:
  - Register values are restored from the stack
  - The PC is loaded with the return address that was also stored on the stack
  - The stack pointer is updated to point to the previous stack frame



© University of Melbourne 2025

Memory Boundaries

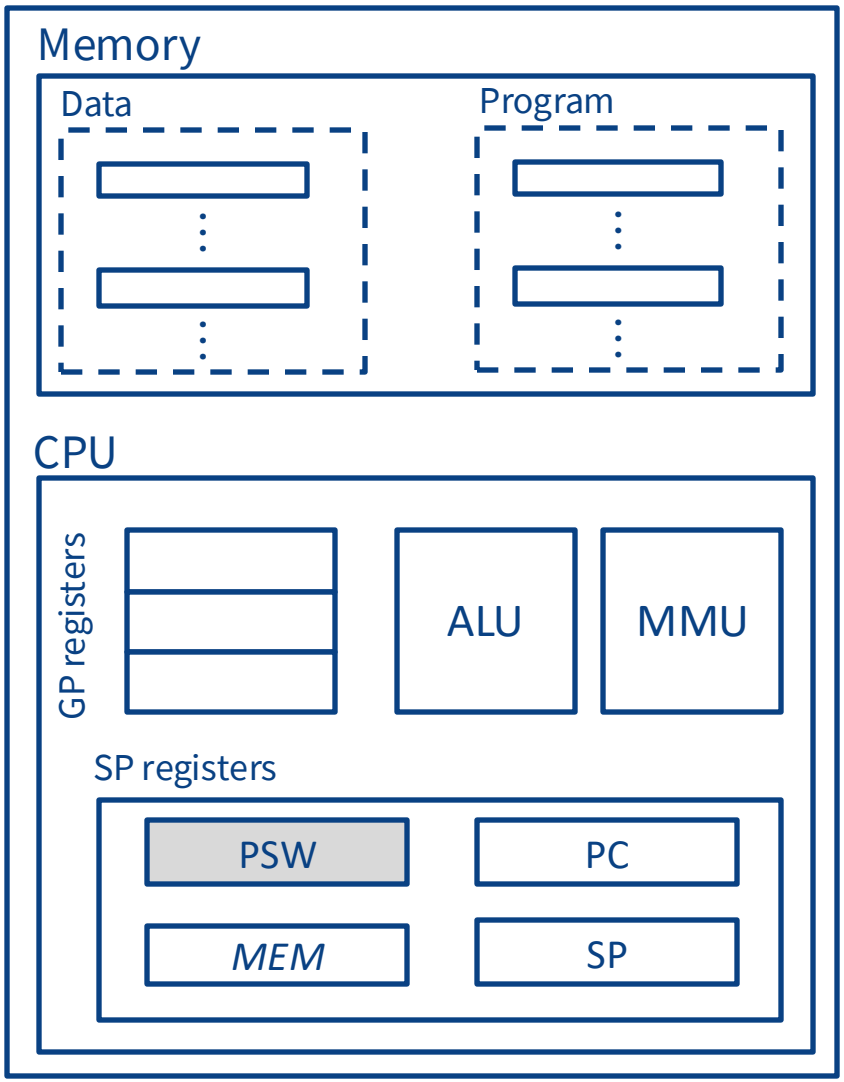
- HW support to enforce memory boundaries between processes
- Allows the operating system to prevent one application from modifying the memory of another and to protect itself



© University of Melbourne 2025

Execution Mode

- Mode **bit** in Program Status Word (PSW) indicates the execution mode of a program
- 2 execution modes: **kernel mode** and **user mode**



© University of Melbourne 2025

User vs. Kernel Mode

- Code running in user mode:
  - Cannot issue privileged instructions
  - Can access only the parts of memory allowed by the OS
- Code running in kernel mode:
  - Can issue all instructions
  - Can access all memory
- Privileged instructions
  - Instructions that affect control of the machine or do I/O

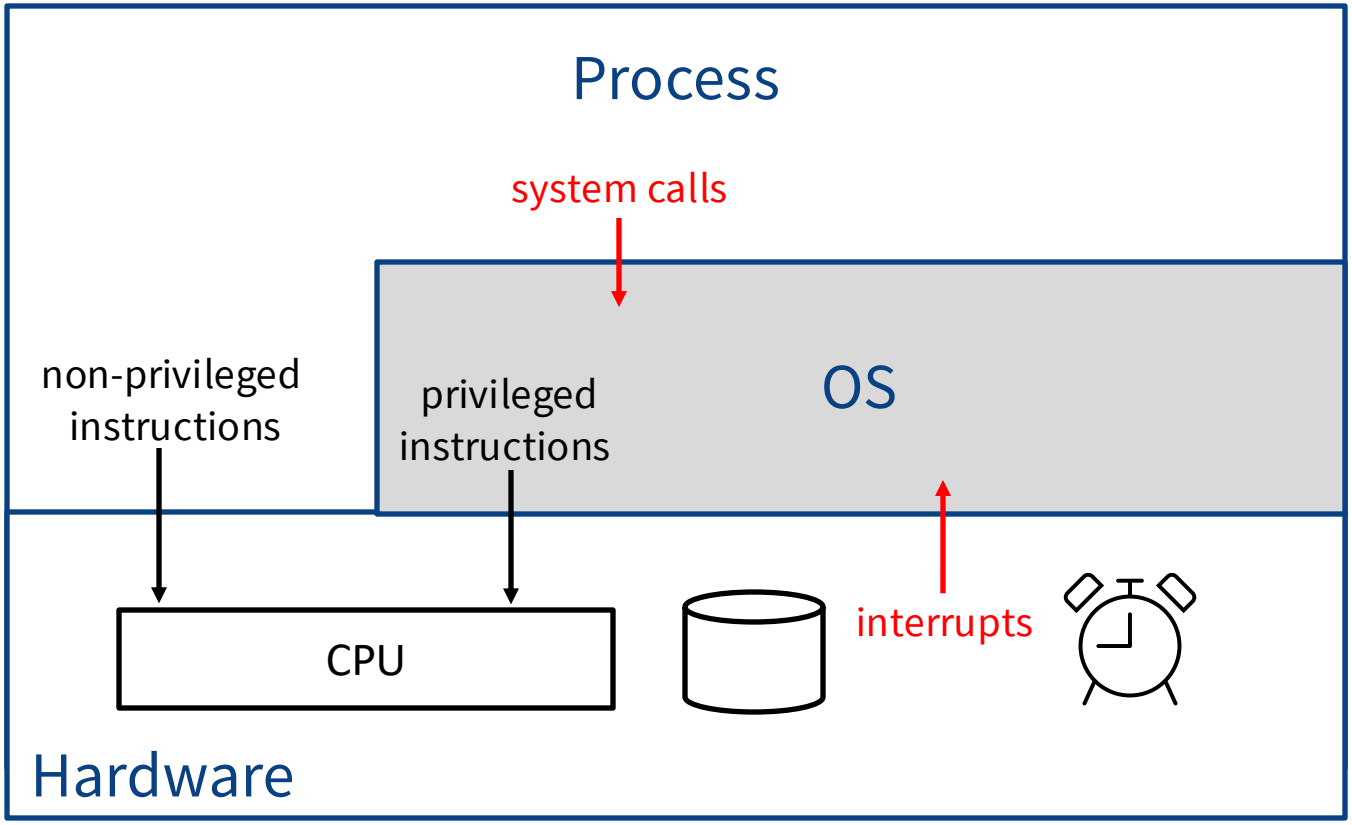
© University of Melbourne 2025

OS Kernel

- Part of the OS that runs in kernel mode
- The core of the OS: a collection of functions that perform management services such as scheduling, resource allocation, and accessing I/O devices.

© University of Melbourne 2025

Basic Operation of an OS



© University of Melbourne 2025

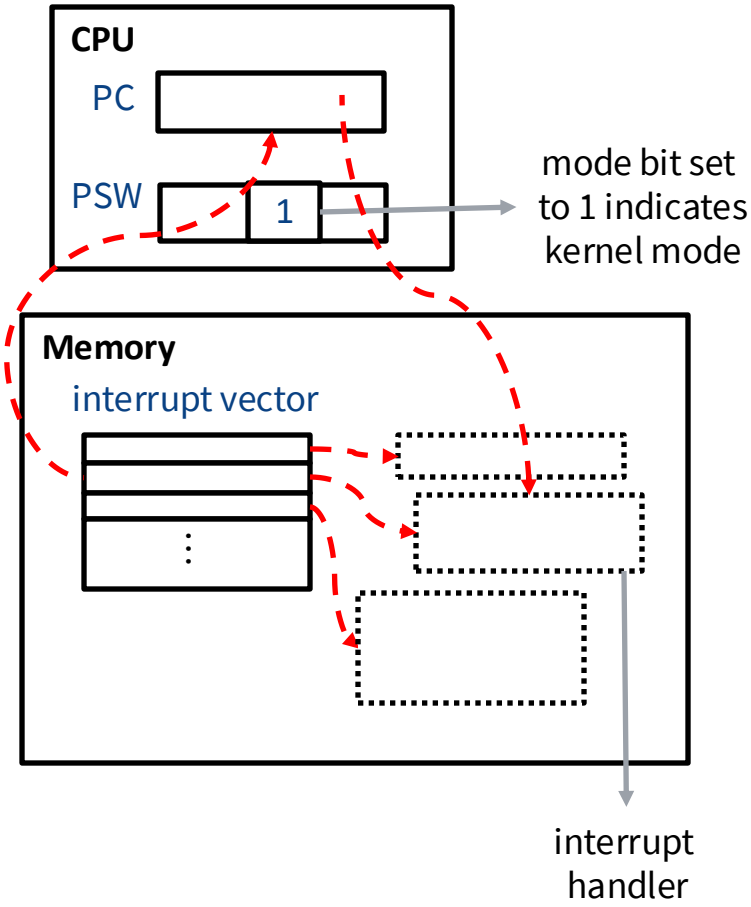
Interrupts

- Events that cause the CPU not to execute the next instruction
- CPU enters kernel mode and executes an OS interrupt handler
- Interrupt vector (in OS memory) stores addresses of interrupt handlers

© University of Melbourne 2025

Interrupts

- When an interrupt occurs, the CPU loads the:
  - PSW – to indicate execution in kernel mode
  - PC – with the address in the specified entry in the **interrupt vector** (address of the **interrupt handler** function)
- When the handler finishes running, the execution of the interrupted application continues where it left off — except if the OS killed the application or decided to schedule another one



© University of Melbourne 2025

Types of Interrupts

- External interrupts – generated by external devices at unpredictable times
  - **Clock interrupt**: Tells the OS that a certain amount of time has passed
  - **I/O device interrupt**: Tells the OS that an I/O operation has completed.
- Internal interrupts – caused by an exception when executing the current instruction. Because the processor cannot complete the instruction, it transfers responsibility to the OS
  - **Error condition**: Tells the OS that the current application did something illegal (divide by zero, try to issue a privileged instruction, etc.)
  - **A temporary problem**: E.g., the process tried to access a page of memory that is not allocated at the moment. This is an error condition that the operating system can handle by bringing the required page into memory

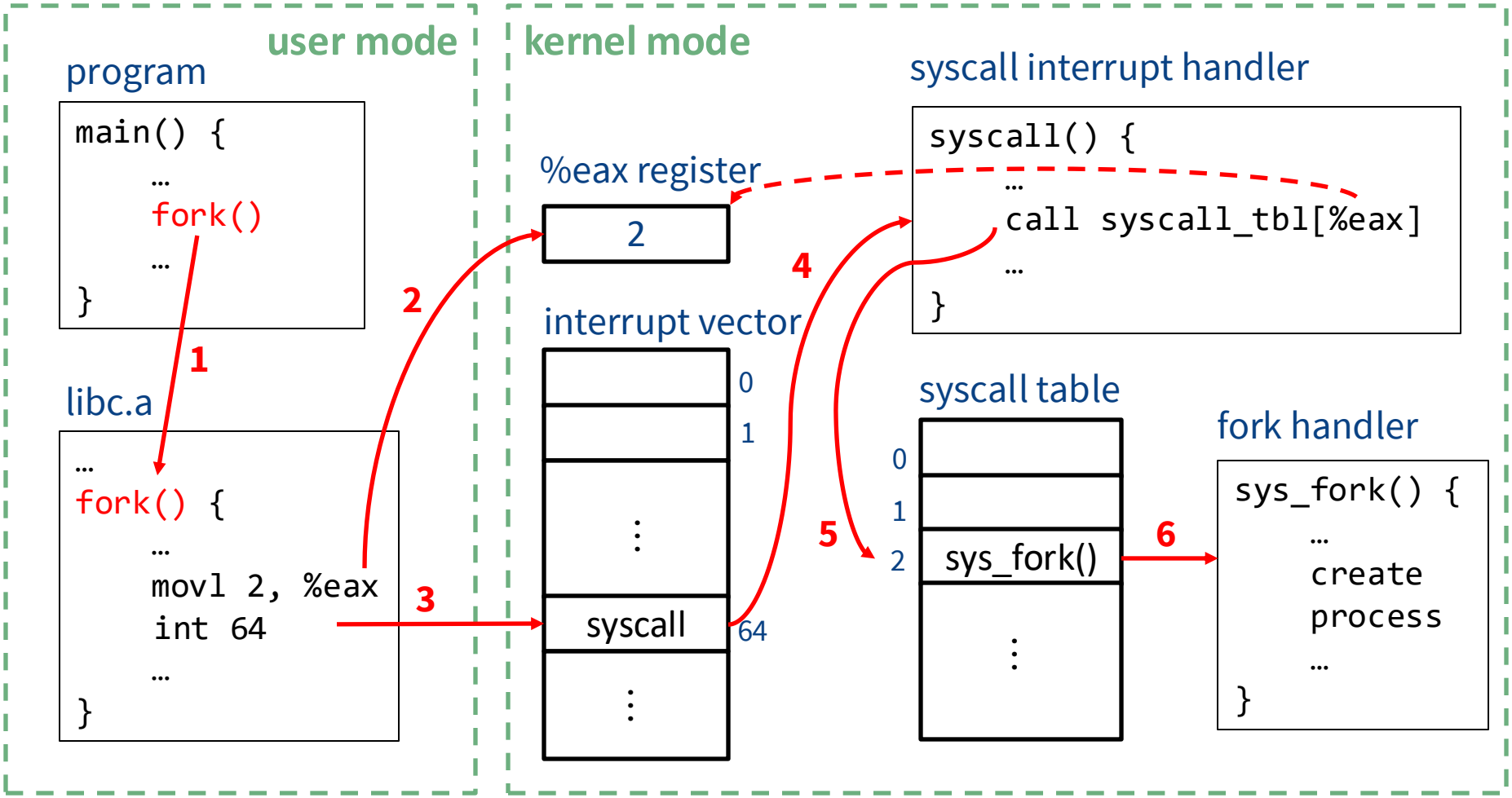
© University of Melbourne 2025

System Calls

- Interface between user programs and the operating system
- System call execution:
  1. Put the system call number in a place where OS expects it (e.g., register)
  2. Execute a TRAP instruction to switch from user mode to kernel mode and start execution at a fixed address within the kernel
  3. The kernel code that starts following the TRAP examines the system-call number and dispatches it to the correct system-call handle
  4. The system call handler runs
  5. Once it has completed its work, control may be returned to the user program
     - The system call may block the caller, the OS will look around to see if some other process can be run next

© University of Melbourne 2025

System Calls



Examples of System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

File management

Call	Description
fd = open(file, how, ...)	Open a file for reading, writing, or both
s = close(fd)	Close an open file
n = read(fd, buffer, nbytes)	Read data from a file into a buffer
n = write(fd, buffer, nbytes)	Write data from a buffer into a file
position = lseek(fd, offset, whence)	Move the file pointer
s = stat(name, &buf)	Get a file's status information

Directory- and file-system management

Call	Description
s = mkdir(name, mode)	Create a new directory
s = rmdir(name)	Remove an empty directory
s = link(name1, name2)	Create a new entry, name2, pointing to name1
s = unlink(name)	Remove a directory entry
s = mount(special, name, flag)	Mount a file system
s = umount(special)	Unmount a file system

Miscellaneous

Call	Description
s = chdir(dirname)	Change the working directory
s = chmod(name, mode)	Change a file's protection bits
s = kill(pid, signal)	Send a signal to a process
seconds = time(&seconds)	Get the elapsed time since Jan. 1, 1970

TB: Figure 1-18

© University of Melbourne 2025